

Ocar — APIs & Flows Under Test

Companion to the load & spike test plan

Version 1.0 | September 2026

Each scenario in the test plan drives a real, ordered sequence of API calls, not isolated hits. Laid out here as flows, in the order each one actually runs, since the sequencing is what makes this a functional test and not just a traffic generator.

1. Rider booking flow — booking_flow, booking_spike

Step	Call	Purpose
1	POST /api/v1/pricing/estimate	Get a fare quote for the trip before booking
2	POST /api/v1/rides	Create the booking
3	GET /api/v1/rides/:id	Read the ride back — confirms the write is immediately readable. Load test only; the spike test skips this step to isolate write-path latency during the surge.
4	POST /api/v1/rides/:id/cancel	Cancel, so runs don't leave thousands of unmatched rides behind in staging

2. Driver live-tracking flow — live_tracking

Step	Call	Purpose
1	POST /api/v1/rides/sessions/online	Driver goes online — the same call the real driver app makes on "Go Online," returns a session id
2	Socket.io location:update event	GPS ping, emitted every 3-5 seconds for the length of the simulated shift, over the same socket path the driver app itself uses
3	POST /api/v1/rides/sessions/offline	Driver goes offline at the end of the shift, so staging doesn't accumulate drivers stuck "online"

3. Rider connection flow — rider_watch, rider_spike

Not a call sequence — a single Socket.io connection, held open and mostly idle, responding only to Engine.IO keep-alive pings. Represents a rider with the app open in the background, which is what actually produces the bulk of the 5,000-6,000 concurrent-connection figure, not booking activity.

4. Ride-accept race — accept-race.js, a separate script

Not a sequence either — a concurrency test of one endpoint. POST /api/v1/rides/:id/accept is fired from several different driver accounts at the exact same ride ID, truly simultaneously. Exactly one call is expected to succeed; the rest must get a clean rejection.

5. Admin dashboard flow — admin_dashboard

Four independent reads, polled on a timer while every flow above runs concurrently — this is what tests read/write contention, not read performance in isolation:

Call	Purpose
GET /api/v1/admin/rides/stats	Aggregate ride counts/status the ops dashboard header shows
GET /api/v1/admin/rides?limit=20	The rides list table
GET /api/v1/admin/sessions/active	Live-map data source — every currently active driver session
GET /api/v1/admin/drivers/:id/rides	One driver's ride history, as an ops admin would open from the driver detail view